EECS1015 – Final Exam Python Cheatsheet

Important functions

```
input(), print(), str(), int(), float(), len(), type(), range(start, end, step)
random.randint(min, max), math.sqrt(y)
```

Escape characters

```
\n (newline), \t (tab), \\ (slash), \" (double quote), \' (single quote)
```

String methods

```
upper(), lower(), capitalize(), title(), find(x), replace(x,y), strip(), count(x)
```

String index/slice syntax

```
string[index]            string[start:end]
string[:end]             string[start:]
```

Formatted printing

```
% method            placeholder syntax  %[width][.precision][type]
.format() approach  placeholder syntax  {:[width][.precision][type]}
f"string"           placeholder syntax  {variable:[width][.precision][type]}
Types: d, f, s
```

If-statements syntax

```
if <condition>:          if <condition>:          if <condition>:
      block                    block                    block
                         else <condition>:        elif <condition>:
                               block                    block
                                                [elif <condition>]:
                                                      block
                                                [else:]
                                                      block
```

While/for syntax

```
while <condition>        for variable in range(..):      for variable in collection_var:
      block                    block                           block
```

```
Keywords: pass, continue
```

Function syntax

```
def func(parameters…):
      [global variables]
      block
      [return …]

def foo(parameters…):                  # example of return more than 1 item
      [global variables]
      block
      return item1, item2              # return two items

x, y = foo()                           # calling function with multiple returns
```

Syntax for declaring lists, sets, tuples, dictionary variables

```
list_var = [item1, item2, item3]             tuple_var = (item1, item2, item2)
list_var = [] # empty list                   tuple_var = ()  # empty tuple
dict_var = {key1:value1, key2, value2}       set_var = {item1, item2, item3}
dict_var = {} # empty dict                   set_var = set() # empty set
```


Accessing/Deleting items from lists, sets, dictionary

```
dict_var[key] = value     # assigns value at key (if key doesn't exists, creates it)
dict_var[key]             # returns value at key
list_var[index]           # returns value at index
list_var[index] = value   # assigns value at index
set_var.add(key)          # adds an item to the set
set_var.remove(key)       # removes a key from the set
del dict_var[key]         # deletes dictionary entry
del list[index]           # deletes list item (re-adjusts all positions)
```

List slicing (also works on tuples)

```
list_var[:] #makes a copy       list_var[start:end]
list_var[:end]                  list_var[start:]
list_var1 + list_var2 # returns new concatenated list
```

List/Tuple methods (* means also applies to tuples)

```
append(item) # adds item to the end    insert(index,item)  # inserts item at position index
clear()      # deletes all items       index(item)*        # find location of item
copy()       # copies the list         remove(item)        # deletes item
reverse()    # reserves the list       sort()              # sort list
extend(list) # extends by list         count(item)*        # returns # of times item appears
```

Dictionary methods

```
values()     # returns values          I     items()      # returns a tuple of tuple (key,value)
keys()       # returns keys                  get(key)     # returns value for key
copy()       # copies the dictionary         update({key:value}) # updates dictionary
clear()      # removes all items
```

Set methods

```
clear()           # removes items             add(item)    # adds item
y.intersection(x) # returns all items that in both y and x
y.union(x)        # return all item that are in both y or x
```

List/Set/Dictionary functions

```
max(collection)           # finds max item
min(collection)           # finds min item
tuple(x), set(x), list(x) # converts collection x to a tuple, set or list
```

String join/split

```
"str".join(list)          # makes a string with "str" inserted between list items
"x,y,z".split("char")     # splits string based on "char" delimiter. Returns a list of items.
```

Misc operators

```
x in collection_var       # checks to see if item x is in collection
id(x)                     # returns id of object x
x is y                    # returns True if x and y are the same object
```

Nested collections (examples)

```
x = [ [1,2], [2,3], [4,5] ]                        # list of lists
y = {"key1":[1,2], "key2":[2,3]}                   # dict of lists
z = [ {}, {}, {}, {} ]                             # list of empty dictionaries
w = {"key1":{"key1":10}, "key2":{"key1":20} }      # dict of dicts
k = [ [[1,2],[2,3]], [[5,6],[7,8]] ]               # list of list of lists
```

Classes

```
def class_name:
    class_var1                  # optional

    def __init__(self, parameters):
       self.instance_var1       # accessing/assigning instance variable
       self.instance_var2       # accessing/assigning instance variable
       class_name.class_var1    # accessing class variable

    def method1(self, parameters):
       ...
       return object            # optional

    def method2(self, parameters):
       x = self.method1(arg1)   # example calling a method within a class method
```

Example usages

```
# declaring class
class foo:
   def __init__(self,x=0):
      …
   def bar1():  # returns a int
      …
   def bar2():  # no return
      …

# declaring objects
foo1 = foo()        # creating object1
foo2 = foo()        # creating object2
y = foo1.bar1()     # calling a method that has a return
foo2.bar2()         # calling a method with no return

# objects and dictionaries
fooDict = { "foo1":foo1, "foo2":foo2, "foo3":foo() }       # creating dictionary of foo objects
y = fooDict["foo1"].bar1()                                 # accessing foo1 object via dict
fooDict["foo3"].bar2()                                     # accessing foo3 object via dict

# objects and lists
fooList = []                    # creating a list of three (3) objects
for i in range(3):
   fooList.append( foo(i) )     # adding object to list

for foo_obj in fooList:         # using the objects in the list
    y = foo_obj.bar1()

for i in range(3):              # using objects in the list via position list indexing
    fooList[i].bar2()
```